As mentioned, there are three reasons to construct logic that controls message flow in your applications. Let’s dig a little deeper into them:

- *Performance*—If a task takes a lot of time and you have many such tasks, although they can be processed in parallel, they should be split between different instances. In the speeding ticket example, drivers can be evaluated in parallel because all the processing logic is contained exclusively in each recorded case.
- *Content of the received message*—The message has an attribute (in this example, speed). Depending on its value, the message should go to one task or another.
- *State of the router*—For example, if the photo processor is busy, all new messages must be sent back to the camera; otherwise, they should be processed normally.

In all cases, the application must decide based on a certain logic which task (actor) it should send a message to. The possible actors a router can choose from are called *routees* in Akka.

In this chapter, you learn about various approaches to routing messages and some other Akka mechanisms that are useful for implementing routers and your own processes. One example is processing messages differently depending on the state of an actor that receives them. The next section covers routing by message content or the actor state.

6.3 Balancing load using built-in routers